

An explainable deep learning-enabled intrusion detection framework in IoT networks

Marwa Keshk^a, Nickolaos Koroniotis^a, Nam Pham^a, Nour Moustafa^{a,*}, Benjamin Turnbull^a, Albert Y. Zomaya^b

^a University of New South Wales (UNSW Canberra) Canberra, 2612, ACT, Australia

^b School of Information Technologies, University of Sydney, NSW, 2006, Sydney, Australia

ARTICLE INFO

Keywords:

Intrusion Detection System (IDS)
Artificial Intelligence (AI)
Explainable AI (XAI)
Internet of Things (IoT)
Cyber defence

ABSTRACT

Although the field of eXplainable Artificial Intelligence (XAI) has a significant interest these days, its implementation within cyber security applications still needs further investigation to understand its effectiveness in discovering attack surfaces and vectors. In cyber defence, especially anomaly-based Intrusion Detection Systems (IDS), the emerging applications of machine/deep learning models require the interpretation of the models' architecture and the explanation of models' prediction to examine how cyberattacks would occur. This paper proposes a novel explainable intrusion detection framework in the Internet of Things (IoT) networks. We have developed an IDS using a Short-Term Long Memory (LSTM) model to identify cyberattacks and explain the model's decisions. This uses a novel set of input features extracted by a novel SPIP (S: Shapley Additive exPlanations, P: Permutation Feature Importance, I: Individual Conditional Expectation, P: Partial Dependence Plot) framework to train and evaluate the LSTM model. The framework was validated using the NSL-KDD, UNSW-NB15 and TON_IoT datasets. The SPIP framework achieved high detection accuracy, processing time, and high interpretability of data features and model outputs compared with other peer techniques. The proposed framework has the potential to assist administrators and decision-makers in understanding complex attack behaviour.

1. Introduction

Emerging technologies, such as the Internet of Things (IoT), have enabled the development of vast interconnected and intelligent environments, such as smart cities. Ubiquitous IoT devices provide continuous environmental feedback and allow the automated adjustment of related systems, fundamentally changing how modern society operates. However, due to added complexities, mis-configurations and conflict between modern and legacy systems, securing such emerging technologies has become a considerable challenge, significantly impacting the stability and availability of existing infrastructure [1]. Such systems comprise a wide variety of low-cost devices and generate significant quantities of data across multiple networks, protocols and for different use-cases [2]. Due to the proliferation of heterogeneous IoT devices and the resulting high-dimensional and multi-modal data they generate, big

* Corresponding author.

E-mail addresses: marwa.keshk@unsw.edu.au (M. Keshk), n.koroniotis@unsw.edu.au (N. Koroniotis), thenam.pham@student.unsw.edu.au (N. Pham), nour.moustafa@unsw.edu.au (N. Moustafa), benjamin.turnbull@unsw.edu.au (B. Turnbull), albert.zomaya@sydney.edu.au (A.Y. Zomaya).

<https://doi.org/10.1016/j.ins.2023.119000>

Received 23 October 2021; Received in revised form 4 August 2022; Accepted 21 April 2023

Available online 28 April 2023

0020-0255/© 2023 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

data techniques are required to effectively analyse data obtained from IoT networks. Artificial Intelligence (AI) is one paradigm that is leveraged to help solve these issues, as AI technologies, especially Machine Learning (ML) and Deep Learning (DL), have been utilised across industries and achieve excellent performance with large scales of data.

IoT networks have unique properties that require unique approaches for defence than those used in traditional corporate environments. Multiple security mechanisms have been proposed to prevent malicious activities and protect IoT networks. Traditional approaches, such as anti-malware, firewalls, user authentication and data encryption, are all well-known. Each is designed for and best suited to promoting security in distinct situations [3,4]. However, such mechanisms are less effective as they are less flexible and dynamic in the face of the rapid growth of attack techniques [5]. Intrusion Detection System (IDS) is a mature and prominently used cybersecurity detection control that can identify a diverse range of well-established as-well-as zero-day cyberattacks [6]. Due to the data-driven adaptive nature of AI techniques, AI-based IDS can achieve high performance with benchmark datasets. Deep learning (DL) techniques can analyse complex data and learn from previous attack patterns to detect zero-day (i.e., new) attacks, accomplishing high performance when processing vast quantities of data sources. However, such techniques typically have a high false-positive rate, and their decisions are opaque to users [7].

Contemporary IDS rely on a range of detection mechanisms, a critical example of which is anomaly-based detection, which is best suited for discovering zero-day attacks and can be utilised to protect IoT systems [8]. On the other hand, current signature-based IDS rely on known attack signatures and suffer from high false positive and false negative rates, limiting their ability to detect unknown attacks and hindering their practical use [9]. Such signature-based systems are known to be efficient but are susceptible to bypass techniques by skilled attackers. Additionally, signature-based IDS platforms cannot discover new or zero-day attacks, as they are limited to signatures for attacks that have been previously analysed and are stored within a database [10]. Thus, the development of adaptable and effective anomaly-based IDS, has driven researchers to evaluate numerous ML models, aiming to achieve high accuracy and low false-positive rates.

Furthermore, the need to explain the predictions of ML-based IDS arises as different types of users are benefited from understanding the root cause of intrusion detection. The field of explainable artificial intelligence (XAI) has been coined to address the growing need to interpret ML models in cyber defence applications. XAI illuminates the black-box model by giving explanations of its functioning and predictions. One of the critical limitations of existing DL-based IDS models is the discovery of zero-day attacks and interpretation of how the models successfully discovered them [5,7]. Researchers have focused on interpreting machine learning models to explain the models' decisions [11]. The purpose of XAI-based intrusion detection systems (IDS) is to devise models' transparency and enhance users' trust in black-box approaches in IoT networks. The explanation challenges arise due to the increased complexity of IoT architectures and the cyberattacks that target them. Moreover, the heterogeneity of data that do not conform to uniform standards consists of a collection of visual, audio and text data, often in a manufacturer-preferred format.

This paper intends to promote the interpretability and explainability of IDS. We propose the novel SPIP (S: Shapley Additive exPlanations, P: Permutation Feature Importance, I: Individual Conditional Expectation, P: Partial Dependence Plot) framework. The proposed framework provides both local and global explanations to serve different purposes. The proposed explanations are intuitive and useful for end-users, security experts and researchers. The proposed framework aims to promote the interpretability and explainability of IDS and enhance their performance. The novelty of the proposed SPIP framework, compared to the existing literature, lies in the combination of local and global explanations for improving the trustworthiness and performance of IoT-enabled IDS. In the context of local explanations, SHAP generated values for each feature, dictating how a single record's characteristics affected the IDS model's output. Global explanations using PFI in the proposed framework consider the entire dataset as input, identifying previously unknown benign and malicious instances in the model.

The major contributions of this paper are structured as follows:

- We propose a novel explainable deep learning-based intrusion detection system (IDS) that provides global and local explanations to IDS regardless of the underlying algorithm used.
- We employ a set of the essential features extracted by the proposed framework and the original set of input features to train the AI-based IDS and compare the performance. The utilisation of input features increases the performance and reduces the training time as well as the total detection time of the IDS.
- We demonstrate the proposed intrusion detection framework's ability to effectively enhance the interpretability and explainability of cyber defence systems in IoT networks.

The remainder of this paper is organised as follows. Section 2 discusses the related works. The methodology of this framework is described in section 3, including Long Short-Term Memory (LSTM), Shapley Additive Explanations (SHAP), Permutation Feature Importance (PFI), Individual Conditional Expectation (ICE) and Partial Dependence Plot (PDP). Section 4 proposes the proposed SPIP framework. Section 5 presents the experiments and the results discussions. Finally, section 6 summarises this work and discusses the future research direction.

2. Literature review

This section presents the background and related work of intrusion detection, explainable AI, and explainable AI-based IDS in IoT networks.

2.1. Intrusion Detection Systems (IDS)

Originally introduced in 1987 by Denning [12], Intrusion Detection Systems (IDS) are a prominent control in cyber defence and are used extensively to detect and prevent cyberattacks effectively. There are various IDS approaches, with network-based IDS and host-based IDS being the two main types, categorised based on deployment area. Whilst network-based IDS detect attacks by capturing and analysing network traffic between devices, host-based IDS monitor processes and system events on the host to identify malicious activities [13].

An IDS can be designed based on three detection methods: signature-based, anomaly-based or hybrid of the two [14]. Signature-based IDS rely on expert-derived rules and pre-constructed attack signatures utilised as a baseline to compare with recorded observations of a system/network and flag any events as potential malicious cyber activities. Anomaly-based IDS rely on automated ML/DL models trained to identify legitimate user operations in a system, thus flagging any inconsistent abnormal behaviour as a potential cyberattack. Although signature-based methods perform better than anomaly-based IDS when tasked with detecting known and well-established cyberattacks, they are susceptible to obfuscation tactics that alter a cyberattack's behaviour or signature (e.g. mutating malware). In contrast, anomaly IDS can detect altering and zero-day cyberattacks, as they focus on modelling legitimate actions. A combination of the two methodologies of signature and anomaly-based event detection results in hybrid IDS.

2.2. Explainable Artificial Intelligence (XAI)

Explainable Artificial Intelligence (XAI) was first introduced in the 1980s [15]; however, it has not been the focus of research, as AI's growth has instead focused on predictive performance. In recent years, the need for XAI has increased as AI models are widely used in critical sectors where explanations for their reasoning and output are required. Furthermore, XAI can enhance the use of AI models as it aids researchers in model debugging, data collection, trust-building and human decision-making. There are two main approaches of XAI methods; intrinsic XAI and post-hoc XAI. Intrinsic XAI approaches explain the structure and functioning of the model, limiting its application to a specific type of AI method. Post-hoc approaches explain the model's final decision by analysing the set of input and the set of output; thus, it is applicable to any model. A further classification of XAI includes the global and local methods, where global XAI aims to explain the models' general characteristics by analysing all of its decisions, whilst local XAI methods focus on giving explanations to individual decisions the model has made.

The large scale development of the Internet of Things (IoT) has caused significant new challenges in cyber defence. IoT deployments are comprised of a large number of Internet-connected devices that generate vast amounts of traffic and digital data, which challenge the ability of existing solutions to detect and prevent cyberattacks [16]. As a result, AI-based IDS have emerged as a prominent method to deal with such large-scale data [17]. A considerable body of contemporary research has focused on developing and evaluating the best IDS models for IoT environments [16]. Due to these challenges and the need to understand the reasoning of automated AI-based models, enhance their performance, and promote interpretability and explainability, XAI methods have been considered additional modules for AI-based intrusion detection models in heterogeneous IoT networks.

XAI methods have been designed and applied to various applications and fields. The most popular works focus on model explanations for computer vision [18], Natural Language Processing (NLP) [19] and voice analysis [20]. Of particular note, the authors of [19] developed a framework, LIME, in the field of NLP and [21] used SHAP to give explanations in the field of biology. In contrast, the utilisation of XAI methods in cyber defence is underdeveloped. In cyber defence, AI is mainly utilised to build a model for automated detection, with numerous research papers have shown the promise of AI-based IDS. Of note amongst these, Rahul et al. [17] used a benchmark dataset to compare the performance of several classical ML algorithms and Deep Neural Network (DNN) models. In this experiment, AI-based IDS obtained excellent performance, particularly those built upon DNNs outperformed other classical ML algorithms.

Andalib and Vakili [22] proposed an IDS combining three different types of ML techniques. Based on their observations, ML models have a short training time and only need low computational resources [22]. The sub-domain of DL consists of a multitude of diverse models, such as Multi-Layered Perceptrons (MLP), Convolutional Neural Networks (CNN), Deep Belief Networks (DBN), Generative Adversarial Networks (GAN), Recurrent Neural Networks (RNN) and their variant Long-Short Term Memory RNN (LSTM-RNN). Different applications, and thus data, place different requirements on the DL model architecture, and with IDS, models that can capture time-series relationships, such as RNN and LSTM, are required. However, despite their superior performance, the AI-based intrusion detection models are becoming more complex, and their functions and outcomes are difficult to interpret. This renders them black-box models and hinders their effective application in critical sectors, including cyber defence.

2.3. XAI-based IDS in IoT networks

Several works related to explainable AI-based IDS in IoT have emerged to confront this problem. Islam et al. [23] proposed a method to gather and use domain knowledge to automate the defence response and improve the explainability of the IDS model. In the experiment, domain knowledge (including the fundamental principle of Confidentiality, Integrity, and Availability (CIA)) is instilled into the ML models. Results from this experiment highlighted increased generalizability and explainability of the models, so the utilisation of AI-based IDS is promoted in IoT networks. Marino et al. [24] produced explanations for incorrect samples using an adversarial approach. This approach works by finding the minimum modification to correctly label the inaccurate records and then visualising the most critical features to explain the model's wrong decision.

As part of XAI, Kristijonas et al. [25] investigated the applicability of concepts and principles from the computational argumentation domain, with its various argumentation frameworks and established reasoning algorithms. The researchers compiled a concise survey of existing argumentation-derived reasoning methods and how they can be leveraged to build explanations, describing systems that are explainable by design, where explanations are called *intrinsic*. Other cases where existing systems are outfitted with argumentation frameworks and the explanations are called *posthoc*. The researchers have linked computational argumentation with explanations and provided a roadmap for future development of argumentation framework-based XAI, however the presented work remains theoretical. The discussion lacks details on how the explanations would be incorporated into the DL models leveraged in cybersecurity applications.

Navid et al. [26] developed a system with a conversation-like interface to address the limitations of existing XAI methods that generate explanations based on feature participation in a model's output and produce graphics that are hard to interpret by regular users. The proposed system is model-agnostic, treating ML/DL models as black-boxes and explainer-independent, leveraging a natural language understanding module for interpreting user input and a natural language generation module for producing valid responses. In contrast, a dialogue state tracker module manages the user-to-module conversation. The proposed system is promising, as it addresses concerns with XAI's complexity that often hinders its primary objective of rendering models transparent to users; however, the system components are not adequately analysed. Furthermore, the effectiveness of the proposed system and the quality of explanations generated by it are not fully addressed.

In a survey by Keane et al. [27], 100 XAI methods that provide counterfactual explanations were investigated in both a psychological and computational context to identify their benefits and evaluate their shortcomings. The researchers assert that end-users can leverage more tangible information and gain insights from the ML models by focusing on counterfactual explanations. In their investigation, the research team identified five shortcomings of existing XAI literature: a lack of user-based interpretability evaluation, ambiguous plausibility of counterfactual explanations, non-consistent explanation sparsity, and non-trivial out-of-scope explanation generation rate and a lack of comparative testing of counterfactual explanations. Although the survey provides valuable insight into the existing literature on counterfactual explanations, the authors neglected to offer a rudimentary evaluation for their proposed roadmap suggestions. Furthermore, the work does not discuss how these explanations could be leveraged by researchers and experts for model tuning and debugging.

Although considerable work in the literature seeks to introduce transparency and explainability in the domain of IDS through XAI, limitations and open challenges still need to be addressed. Whilst the XAI-derived explanations sometimes are complex, hindering interpretability, explanations remain underutilised in other studies, as only a single type of explanation was leveraged [23,28]. Incorporating XAI in IDS designed for IoT networks introduces further challenges. This is because of the heterogeneity of data produced by the range of sensors and actuators that constitute an IoT deployment, coupled with the complexity of recent cyberattacks that target such systems, corresponding to an increase in the complexity of ML/DL predictive models.

To overcome the limitations present in established AI-based IDS, in this paper we propose the SPIP framework that, unlike previous works, combines local and global explanations to enable transparency, promote trust, and allow model bias detection and debugging to address the prevalent limitations and challenges in XAI applications for cybersecurity. This framework combines several XAI methods, including SHAP (SHapley Additive exPlanations), PFI (Permutation Feature Importance), ICE (Individual Conditional Expectation) and PDP (Partial Dependence Plot), with an LSTM model for the detection of intrusion and justification of results through explanation. The SHAP method, has a solid theoretical foundation in game theory for enhancing the interpretability and explainability of AI-based IDS.

The proposed SPIP framework is innovative compared to existing ML-based IDS approaches. It combines local and global explanations to detect bias, introduced to the IDS model through the training data. It also provides interpretability of the LSTM model's results based on input features and record contribution. To further enhance the performance of the proposed IDS framework, we considered the sequenced, multi-staged nature of cyberattacks (like the cyber kill-chain) and selected LSTM, a sequential DL model, as the base model for detecting intrusions in IoT environments. An LSTM's memory can be effectively utilised to detect cyberattacks, as previous states are considered before a new prediction is made. The proposed framework was further utilised to study the behaviour of the trained LSTM model and analyse the benchmark datasets in order to enhance the performance of automated detection in cyber defence systems.

3. Preliminaries

This section introduces the underlying mathematical models of the IDS and SPIP framework. Long Short-Term Memory (LSTM) - a modified version of Recurrent Neural Networks (RNN) is utilised to analyse complex and large-scale datasets to detect cyber attacks. The proposed framework applies various XAI methods to produce reasonable and intuitive explanations that could be employed to study how the model functions, the quality of datasets and the characteristics of many attack classes.

3.1. Recurrent neural networks

Before discussing LSTM, we briefly describe RNN to illustrate the improvements of LSTM models over their original counterparts (RNNs). RNN consists of multiple layers with a feedback loop, making RNN effective in analysing sequence data. Each neuron in RNN has a short-term memory to pass information to itself in future steps. Combining the internal memory and feedback loop helps link the current output with the preceded output, propagating the data from the past to the present. In other words, information travels through time in RNNs; thus, the weights between neurons are updated using backpropagation through time (BPTT) [29]. However,

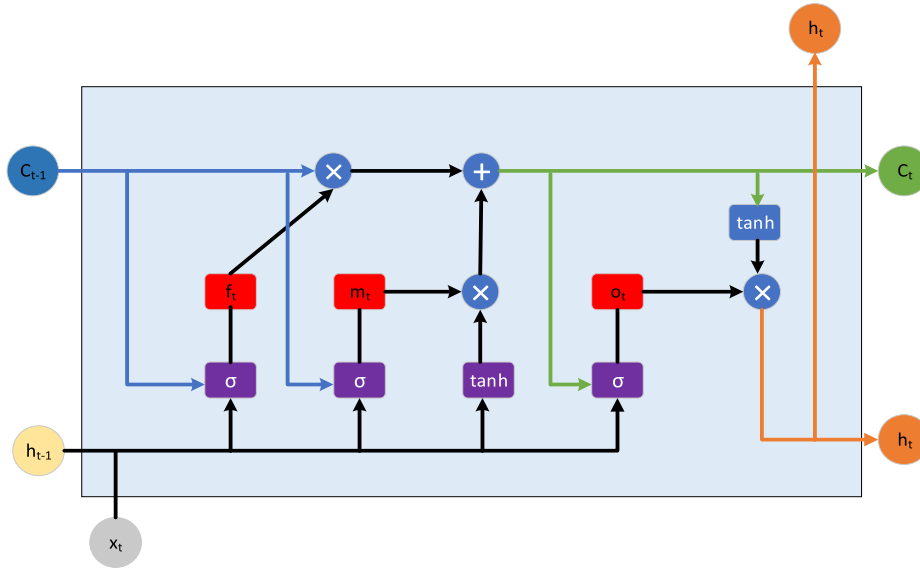


Fig. 1. Long Short-Term Memory.

BPTT suffers from vanishing and exploding gradient problems, as Bengio et al. demonstrated in [30]. Therefore, RNN can only retain information for a few steps and fails to capture long-term data dependencies. Due to the vanishing gradient problem, RNN cannot learn the context information across a long time and become inefficient in extended data sequence analysis.

3.2. Long short-term memory

Long-Short Term Memory (LSTM) is a modified version of RNN that was first introduced in 1997 [31] (Fig. 1). Technically, LSTM prevents the vanishing gradient problem by employing memory cells and gates. The LSTM cell contains three gates: memory, forget, and output gate. The gate mechanism helps LSTM capture long-term dependencies and allows the retention of information for long steps. The gates control the information that flows into and out of the cell state. The memory gate controls the flow of new information. The forget gate deletes information in the cell state. The output gate regulates the output flow of the internal memory cell state.

Firstly, the forget gate decides which information to delete or keep in the cell state by a sigmoid activation function. The sigmoid layer considers h_{t-1} , x_t and c_{t-1} and generates a real number in the range of 0-1 for each number in the cell state c_{t-1} , as shown in equation (1). For example, 0 indicates “delete all information”, and 1 indicates “keep all information”. Secondly, the cell processes new information and decides which information to be stored in the cell state. The sigmoid layer chooses information to update as shown in equation (2) and the $\tanh$ activation function generate a vector of new values. The update to the cell state is the combination of these two values, including information that is recorded in the cell state and the vector of new values generated by $\tanh$ activation function.

Thirdly, after computing values in the first two steps, the new cell state c_t is created in equation (4). It may keep some information from c_{t-1} (decided by f_t) and add new information calculated in the second step. Lastly, the output flow of information is regulated using equation (5). Output gate also uses a sigmoid activation function. The cell state c_t goes through $\tanh$ function, and then it is multiplied by the result from the sigmoid layer.

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f) \quad (1)$$

$$m_t = \sigma(W_{xm}x_t + W_{hm}h_{t-1} + W_{cm}c_{t-1} + b_m) \quad (2)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_{t-1} + b_o) \quad (3)$$

$$c_t = f_t \odot c_{t-1} + m_t \odot \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \quad (4)$$

$$h_t = o_t \odot \tanh(c_t) \quad (5)$$

In the above equations, m_t , f_t , and o_t correspond to memory, forget, and output gate values at time t , respectively. x_t , h_t , and c_t represent the input layer, hidden layer, and cell state at time t . b terms indicate the bias vectors and W terms indicate weight matrices. σ and $\tanh$ are the sigmoid and $\tanh$ activation functions, $\odot$ is the element-wise multiplication.

3.3. Explainable Artificial Intelligence (XAI) methods

Model-agnostic XAI techniques can be classified into local and global explanation methods. Local interpretability utilises a single data record to generate the explanation for a single decision. This explanation can be helpful to end-users by promoting trust in automated model decisions and verifying a single decision. Global interpretability requires the entire dataset as input to generate explanations, demonstrating the black-box model's overall structure and function. This technique benefits researchers and developers who wish to analyse and detect any bias to improve the model's performance. Moreover, in the case of IDS deployments, global XAI methods are important in studying and analysing different types of cyberattacks based on network traffic features.

In this paper, different XAI methods are used to study the functioning and predictions of the proposed IDS locally and globally. On the one hand, we apply Shapley Additive Explanations (SHAP) [32] and Individual Conditional Expectation (ICE) [33] techniques for visualising individual predictions made by the model. On the other hand, Permutation Feature Importance (PFI) [34], SHAP [32], Partial Dependence Plot (PDP) [35] are utilised as global XAI methods to find the customised set of relevant input features, which can be used to explain the functioning of the proposed model as a whole. Those XAI techniques are discussed as follows.

3.3.1. Shapley Additive Explanations (SHAP)

Lundberg et al. [32] proposed SHAP based on the game's theoretically optimal Shapley values in [36]. SHAP has a solid theoretical foundation, which is a combination of Local Interpretable Model-agnostic Explanations (LIME) [37] and Shapley value [36]. Thus, we will briefly introduce LIME and Shapley's value before analysing SHAP. LIME derives from the concept of local surrogate models. In the concept of surrogate model, an interpretable model is chosen to be trained and approximate the predictions of the black-box model to be explained. Unlike global surrogate models, LIME's key concept is fitting an interpretable model around a specific instance to visualise significant features of that data instance in contribution towards an attack or normal prediction of the model. In LIME, the surrogate model can be of any interpretable model as long as it is a good approximation of the black-box model's predictions locally [37]. In LIME, the explanation of an observation x is computed by the following formula:

$$E(x) = \underset{g \in G}{\operatorname{argmin}} \{ L(f, g, d^x) + \Omega(g) \} \quad (6)$$

In equation (6), g denotes an interpretable model and G represents the class of all possible explanation models. These can be any interpretable ML whose results can be understood by a human. The original model is denoted by f and the distance between the sample, and original data is presented as d^x . Thus, $L(f, g, d^x)$ is the loss function that measures the difference in decisions of the surrogate model and the original model. LIME aims to calculate this loss function L . Additionally, $\Omega(g)$ denotes the complexity of model g and it is defined by human. For example, $\Omega(g)$ can be point for the depth of the decision tree model [37].

The LIME algorithm begins by creating non-existing data samples around the chosen instance of interest x . Then a local surrogate model g is trained on this data samples as the loss function L and the complexity Ω are minimized. Eventually, a local model denoted by $E(x)$ can explain the given decision of instance x . The Shapley value is a technique in the cooperative game theory method. The value was named after its inventor Lloyd Shapley as he introduced it in [36]. Shapley value has a strong theoretical foundation to measure the importance of each player in a collaborative game and indicate how much each player has contributed to the success. Specifically, each feature's value has its own Shapley value representing its average contribution to the model's outcome. This value is computed and summed over all combinations of feature values for a given set of features and calculated predictions:

$$\phi_j(val) = \sum_{S \subseteq M \setminus \{j\}} \frac{|S|!(m - |S| - 1)!}{m!} [val(S + j) - val(S)], \quad j = 1 \dots m \quad (7)$$

where:

- S is the subset of features used and $|S|$ is its number.
- M is the complete set of features and m is its size.
- j is the j^{th} feature the vector of the feature values of the instance to be explained
- $val(S)$ and $val(S+j)$ are the functions assigning only subset of features S and subset of features S with j^{th} feature present, respectively.

The Shapley value has four meaningful properties; **Efficiency**, **Symmetry**, **Dummy** and **Additivity**. The property of **Efficiency** states that the feature contributions must add up to the prediction for a single instance, which equals to the difference between the actual output and the average prediction:

$$\sum_{i=1}^m \phi_i(val) = val(M) \quad (8)$$

Symmetry: Considering two different features i^{th} and k^{th} in the subset of features S , so that:

$$val(S + i) = val(S + k), \quad S \subseteq M \setminus \{i, k\} \quad (9)$$

Equation (9) indicates that two features i^{th} and k^{th} contribute equally to all possible coalitions. Therefore, the contribution of two features should be equal and the function ϕ is symmetric:

$$\phi_i(val) = \phi_k(val) \quad (10)$$

Dummy: if the contribution of a given feature does not change the prediction, its Shapley value should be equal to zero:

$$val(S + i) = val(S), \quad S \subseteq M \setminus \{i\} \quad (11)$$

then:

$$\phi_i(val) = 0 \quad (12)$$

Additivity: the gain from combining two functions val and val' is equal to sum of individual gains from each function for every feature i^{th} :

$$\phi_i(val + val') = \phi_i(val) + \phi_i(val') \quad (13)$$

SHAP was first introduced in 2017 [32] as a unified approach for explaining black-box models' predictions. SHAP is a relevance-based approach that measures the importance of each feature by computing its contributions to the model's decision. SHAP can connect LIME and Shapley value by representing Shapley value explanation as a linear model. SHAP value can be calculated using individual data instances. For example, an explanation for an instance x can be estimated using the following formula:

$$g(z') = \phi_0 + \sum_{i=1}^m \phi_i z'_i, \quad z' \in \{0, 1\}^m \quad (14)$$

where:

- m is the size of all possible coalitions.
- g is the explanation model.
- z' is simplified features. The 1 in z' indicates that features in the new data are the same as those of the original data - the instance z .
- ϕ_i is the Shapley value for feature i^{th} of instance x , hence indicates the impact of feature i on the model's prediction.

A key advantage of SHAP over LIME is that effects are distributed relatively in SHAP, yet the model's behaviour is assumed to be linear locally in LIME. Specifically, LIME weighs the instances based on their distance to the original instance. Meanwhile, instances in SHAP are weighted so that the coalition is in the Shapley value estimation. Shapley compliant weighting is calculated by a method called SHAP kernel [32]:

$$\pi_x(z') = \frac{(m-1)}{\binom{m}{|z'|} |z'| (m - |z'|)} \quad (15)$$

where:

- m is the size of all possible coalitions.
- z' is the total number of present features.

To conclude, the process of calculating SHAP values, for instance x is divided into five steps:

1. Sample data $z'_k \in \{0,1\}^m$, $k \in \{1 \dots K\}$
2. Get prediction for each z'_k . This requires 2 steps, including converting each z'_k to the original feature space, and then applying model $f(h_x(z'_k))$. $f(x)$ represents original model, and $h_x(z') = z$ where $h_x: \{0,1\}^m \rightarrow \mathbb{R}$ is used to get valid data instances from coalitions of feature values.
3. Calculate the weight for each z'_k using equation (15)
4. Train the linear model g by optimizing the loss function L and using Z as the training data:

$$L(f, g, \pi_x) = \sum_{z' \in Z} [f(h_x(z')) - g(z')]^2 \pi_x(z') \quad (16)$$

5. Return ϕ_k as Shapley values and coefficients from the linear model calculated in equation (16).

3.3.2. Permutation Feature Importance (PFI)

PFI is another relevance-based model-agnostic method that computes the changes in prediction error of a fitted model when permuting the input feature values. This technique shows the importance of each feature by breaking the relationship between the feature and the desired output. Therefore, the model's error would increase if the model considers the feature of interest necessary. PFI was first introduced in 2001 by Breiman [38]. Based on this concept, Fisher et al. [34] created a model-agnostic version of the PFI named *Model Class Reliance*.

There are three steps in the PFI algorithm proposed by Fisher et al. (2018). Considering the trained model f , feature matrix X , target vector y and error measure $\mathcal{E}(y, f)$:

1. Compute the original model's error: $e^{orig} = \mathcal{E}(y, f(X))$
2. For each $i = \{1 \dots k\}$:
 - Create X^{perm} by permuting feature i in the data X to break the relationship between feature i and true outcome y .
 - New error value $e^{perm} = \mathcal{E}(y, f(X^{perm}))$
 - Calculate the PFI value for feature i^{th} : $S_{\geq} = \frac{e^{perm}}{e^{orig}}$
3. Sort and present features in descending order of value $S_{\geq}$

3.3.3. Partial Dependence Plot (PDP)

A partial Dependence Plot (PDP) is one of the most widely adopted visualisation-based XAI methods. It was first introduced by Friedman (2001) to show the relationship between feature values and the black-box model outcome by visualising on the plot [39]. PDP is a global model-agnostic approach, which considers all records and demonstrates the global relationship between a specific feature and the prediction.

$$PD(x_s) = E(x_c)[f(x_s, x_c)] = \int f(x_s, x_c) dP(x_c) \quad (17)$$

where:

- f refers to the ML model
- $x_s \in S$ - set of input features for which PDP function is plotted.
- $x_c \in C$ - set other input features f , which means that: $x_c = x \setminus x_s$
- $dP(x_c)$ is the marginal distribution of x_c

According to equation (17), the PDP function marginalise model f outcome over the distribution of set C to shows the relationship between the features in set S and model f output. Another way to compute the partial function $PD(x_s)$ by using Monte Carlo method [40]:

$$PD(x_s) = \frac{1}{k} \sum_{i=1}^k f(x_s, x_c^i) \quad (18)$$

where:

- k refers to the total number of data records
- x_c^i are feature values from the set of features C

Equation (18) demonstrates the average marginal effect of given feature values in a set of features S on the model outcome. Additionally, PDP assumes no correlation exists between features in S and features in C . However, this assumption is likely violated in real life; thus, the function PD would take weak and invalid data points as arguments and demonstrate misleading explanations. Moreover, PDP can have a hidden heterogeneous effect as it only visualises the average marginal effect. Specifically, each data point can have a positive or negative association with the model's prediction; thus, only presenting the average effects of all points would hide such relationships [40].

3.3.4. Individual Conditional Expectation (ICE)

Individual Conditional Expectation (ICE) is calculated similarly to PDP, but it is designed to inspect individual observations of the model. Compared to one line overall in PDP, ICE curves display one line for each record, demonstrating the record's prediction changes when a feature changes. In other words, PDP represents the average of all lines in an ICE plot. PDP covers the heterogeneous relationships created by interactions between features. Therefore, it only works well if these interactions are weak. ICE uncovers such relationships, hence providing more insight than PDP. ICE would take each record in the observations set $\{(x_s^{(i)}, x_c^{(i)})\}_{i=1}^k$ and plot the curve $f_s^{(i)}$ against $x_s^{(i)}$. Thus, the feature's relationship with prediction is shown for k observations, and the average of all curves results in PDP.

4. Proposed methodology

This section demonstrates the proposed framework for an explainable AI-based IDS platform. In IDS, identifying malicious behaviour is only the first step because understanding such a decision is crucial for a solution. An insight into the decision helps administrators identify the part of the network, the part of features and the security policies compromised by attackers [41]. With the information provided by XAI, the IDS operator can give the correct actions, whether to debug the IDS model or apply new security policies to prevent the same attacks in the future.

The proposed framework aims to develop an effective explainable AI-based IDS, where various XAI methods are used to generate both local and global explanations. The local approach focuses on explaining single network traffic that was classified as intrusion;

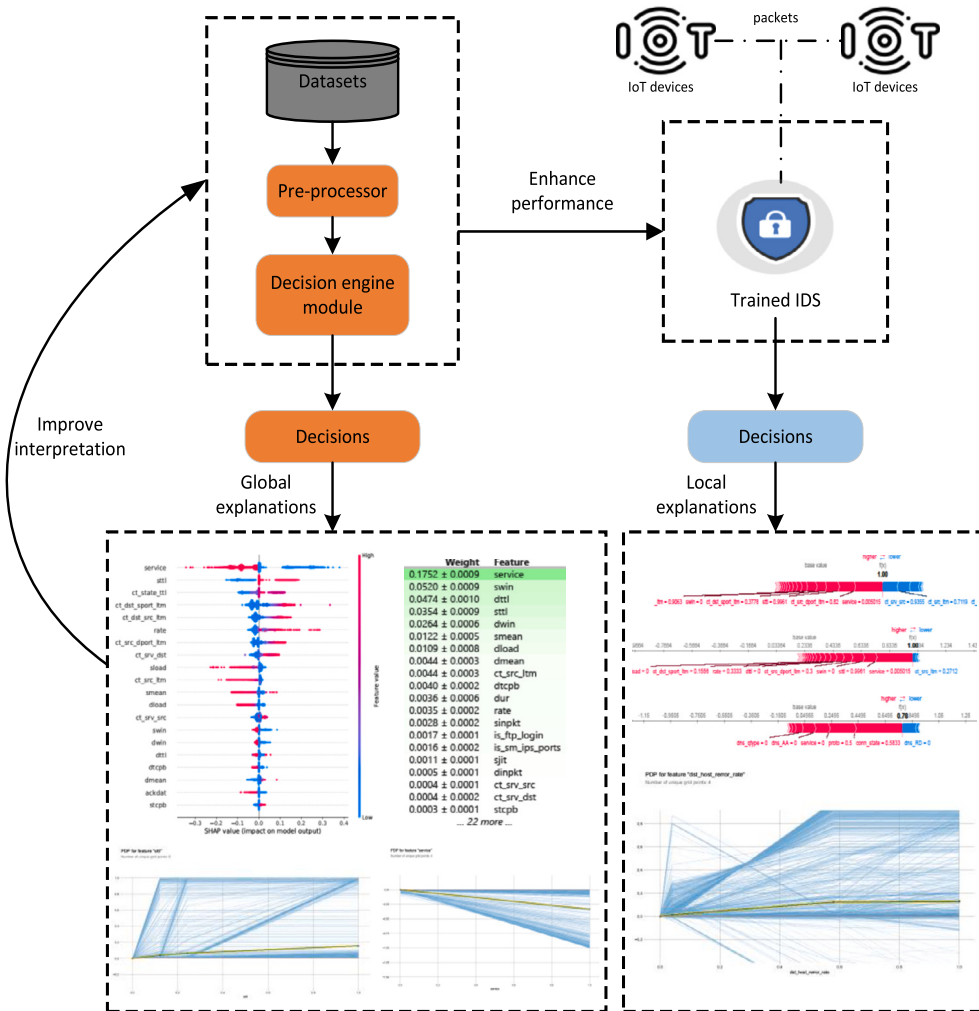


Fig. 2. Overview of the structure of SPIP framework.

thus, it increases the security expert's and user's trust in the IDS. Meanwhile, the global approach gives insights into the complex functioning of the IDS, hence helping the IDS operators to improve the model's efficiency and providing the experts with deep knowledge about cyber attacks. The innovation of the proposed SPIP framework lies in the combination of local and global explanations that enable the description of the inner workings of DL models, thus revealing any previously unknown biases that would have been imparted to the model due to dataset imperfections, while also facilitating the explanation of feature sets and their contribution (Fig. 2).

The selection of an LSTM as the underline model for an IDS is corroborated by the sequential nature of cyberattacks. An LSTM model is designed to handle multivariate and time-series data, such as the network-based datasets selected for this research. This leverages their memory mechanics to manage information retention between time-steps and achieve enhanced performance in terms of detection accuracy and false alarm rate. Considering the Cyber Kill Chain framework [42] as a model for an average Advanced Persistent Threat (APT), the seven defined stages need to be carried out for the attacks to be successful. For example, reconnaissance is the initial stage that provides the necessary information to select the payload, to identify the vulnerability that will be exploited and the delivery method. The datasets selected to evaluate the proposed SPIP framework were developed by leveraging common APT strategies that adhere to the Cyber Kill Chain mode of operation. An RNN-type DL model such as LSTM can be utilised to detect intrusions effectively.

4.1. Local explanation

The proposed framework utilises two different methods to generate local explanations, including Shapley Additive Explanations (SHAP) and Individual Conditional Expectation (ICE). Shapley values of individual prediction measure the effect of each input feature on the model's prediction. The explanations are visualised to be more intuitive. Additionally, XAI recipients can easily find how that value of a particular feature affects the predicted probabilities made by the IDS. Meanwhile, ICE inspects each data record and

displays one line for each instance, demonstrating the prediction's changes according to a specific feature's changes. Therefore, ICE can support Shapley's values very well in showing the impact of a feature on the final prediction.

4.2. Global explanation

The proposed framework utilises three different methods to generate global explanations, including Partial Dependence Plot (PDP), Permutation Feature Importance (PFI) and SHAP. PDP visualises the global impact of specific features on the predictions of a fitted ML model. This helps security experts study each feature's importance in a particular benchmark dataset. PFI computes the change in prediction error of a fitted model over all possible permutations of the feature's values; thus, it shows the global impact of each feature on the model's predictions.

PFI typically shows the twenty most important features from the input features, therefore, it can be helpful in the process of feature selection. Additionally, SHAP can also identify the most essential features by combining Shapley values. Calculating SHAP for each data instance would generate a matrix whose rows represent instances and columns denote features. Then, the importance score of each feature can be obtained by computing the average of the absolute Shapley values per feature across this matrix:

$$\mathcal{A}_j = \sum_{i=1}^k ||\phi_j(x_i)|| \quad (19)$$

where:

- $\mathcal{A}_j$ is the average Shapley value of the j^{th} feature.
- k refers to the number of instances in the dataset.
- $\phi_j(x_i)$ is the Shapley value of the j^{th} feature in the i^{th} instance.

After obtaining these values, the features are typically sorted based on the importance score and the twenty most important features are shown.

5. Experimentation and results

In this section, the experiment configurations, including the datasets used and the structures of the models, are discussed. We also evaluate the performance of different classifiers and the proposed explainable AI-based IDS framework. As discussed above, our framework generates both global and local explanations using multiple XAI methods. The experiments aim to evaluate the efficiency of generated explanations. Additionally, different classifiers are customised to utilise the results obtained from the proposed framework, improving the model's performance and giving more insight into the datasets used and the functioning of the AI-based IDS.

5.1. Datasets

Several statistical analyses have revealed the drawbacks of the benchmark dataset KDD-CUP99 and shown that the dataset may adversely affect the performance, evaluation and comparison of different classifiers. NSL-KDD is a purified version of KDD-CUP99 with significant improvement. Firstly, redundant and duplicate records are removed to prevent the IDS from being biased [43]. Secondly, the number of selected records from each difficulty-level group is inversely proportional to the percentage of records in the KDD-CUP99. As a result, the evaluation of IDS is more accurate because the range of classification rates is wider [43]. Lastly, the NSL-KDD dataset contains a sufficient and reasonable number of records to ensure reliable results from different classifiers. Each data instance has 41 features and one label classifying it into attack or normal network flow. The features can be basic, content-related, time-related or host-based traffic features [44]. There are three subsets in NSL-KDD, including (1)KDDTrain+ for training, (2)KDDTest+ for testing, and (3)KDDTest-21 for advanced testing. Researchers designed 21 machine learning models to evaluate the NSL-KDD datasets. Then, they removed all records in KDDTest+ that were correctly labelled by 21 models and made a new dataset from the remaining called KDDTest-21 [43].

Moustafa and Slay (2015) created the UNSW-NB15 dataset in a synthetic setting built in the UNSW cyber Range lab. UNSW-NB15 is a modern NIDS benchmark dataset as the creators utilize novel methods to generate modern normal and synthetic attack network activities [45]. Unlike NSL-KDD/KDDCup-99 datasets, UNSW-NB15 contains new attack and standard network traffic patterns. The IXIA PerfectStorm platform was utilised to generate network traffic, from which flows were extracted using the Argus and Bro-IDS tools, while twelve algorithms were used to extract a total of 47 features from the generated activities. According to the IXIA tool's report, each record had 2 labels including (1) *attack_cat*: name of each attack category and (2) *label*: 0 for normal, 1 for attack records. UNSW-NB15 has multiple attacks and up-to-date information of packets, which are significant advantages over old datasets.

Moustafa [7] developed a new dataset to validate Industrial IoT (IIoT) systems named the ToN_IoT dataset. The dataset contains telemetry data, which are collected from IoT devices, to detect intrusions that manipulate IoT devices [46]. The IoT telemetry data was generated in a testbed environment with three layers *Edge*, *Fog* and *Cloud* to represent real-life data from contemporary production IoT/IIoT networks. The NSX-VMware platform was utilised to provide the features of Software-defined Network (SDN) and Network Function Virtualization (NFV) to manage the interaction between the three layers [46]. ToN_IoT was generated on a

Table 1
List of symbolic and binary features in the three datasets.

Dataset	Symbolic features	Binary features
NSL-KDD	protocol_type, service, flag	land, logged_in, root_shell, su_attempted, is_host_login, is_guest_login, logged_in
UNSW-NB15	proto, state, service	is_sm_ips_ports, is_ftp_login
ToN_IoT	conn_state, dns_query, ssl_version, ssl_cipher, ssl_subject, ssl_issuer, http_method, http_uri, http_version, http_orig_mime_types, http_resp_mime_types, weird_name, weird_addl, type	dns_AA, dns_RD, dns_RA, dns_rejected, ssl_resumed, ssl_established, weird_notice

Table 2
List of attack types in NSL-KDD.

Class of attack	Attack types
probe	ipsweep, nmap, portsweep, satan, saint, mscan
DoS (Denial of Service)	back, land, neptune, pod, smurf, teardrop, apache2, udpstorm, processtable, mailbomb
u2r (user to root)	buffer_overflow, loadmodule, perl, rootkit, xterm, ps, sqlattack
r2l (root to local)	ftp_write, guess_passwd, imap, multihop, phf, spy, warezclient, warezmaster, snmpgetattack, named, xlock, xsnoop, sendmail, httptunnel, worm, snmpguess

realistic simulation of an IoT environment, incorporating normal and various types of attacks in IIoT applications. For training and evaluating the DL models of SPIP, the default training and testing subsets of each dataset were utilised. Specifically, the number of records for each subset are: 125973 training and 22544 testing records for NSL-KDD, 175341 training and 82332 testing records for UNSW-NB15, 322731 training and 138313 testing records for TON_IoT Network dataset.

5.2. Data pre-processing

Each dataset above has several symbolic and binary input features. Names of such features are listed in Table 1. Symbolic features from these datasets are converted using a label encoder, and binary features remain unchanged. Other input features in the three datasets are continuous, which can be integer or float. In this experiment, we use the min-max normalisation method so that all continuous features are in the range [0,1] after being scaled:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (20)$$

There are 17 classes of attacks in the three datasets. Among those, *Denial of Service (DoS)* attack appears in all three datasets. *Probe* attack in NSL-KDD is equivalent to *Analysis* in UNSW-NB15 and *Scanning* in ToN_IoT. Additionally, both UNSW-NB15 and ToN_IoT have the *Backdoor* attack class. In the NSL-KDD dataset, classes of attack are categorised into different types that are shown in Table 2, with the *label* column corresponding to attack types. Before leveraging the NSL-KDD for training and model evaluation, a conversion of the attack types to attack class is required.

5.3. Evaluation metrics

Accuracy, precision, recall, and F1-score are calculable metrics we selected to evaluate the model's performance. These statistical measures are calculated from ground truth values, including True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN). These evaluation indicators explain the model's performance; thus, they provide feedback to make improvements and design an IDS in IoT networks with desirable performance. The definition of ground truth values are listed as follows:

- **True Positive (TP):** The number of instances correctly classified as the target outcome.
- **True Negative (TN):** The number of instances correctly classified as others.
- **False Positive (FP):** The number of instances wrongly classified as the target outcomes
- **False Negative (FN):** The number of instances wrongly classified as others.

Based on above values, evaluation indicators are calculated as follows:

Table 3
Results about Binary Classifiers.

Dataset	Accuracy	Precision	Recall	F1-score
NSL-KDD	0.811	0.921	0.732	0.815
UNSW-NB15	0.866	0.811	0.988	0.891
ToN_IoT	0.873	0.784	0.880	0.829

1. **Accuracy:** The ratio of the correctly classified instances to the total number of instances in the test set.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (19)$$

2. **Precision:** The ratio of the instances correctly classified as the target outcome to the number of instances classified as the target outcome.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (20)$$

3. **Recall:** The ratio of the instances correctly classified as the target outcome to the total number of target records.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (21)$$

4. **F1-score:** This value measures precision and recall at the same time by using the harmonic mean in place of the arithmetic mean.

$$F1\text{-score} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}} \quad (22)$$

5.4. Experimentation setup

The models were developed using the Python programming language on Microsoft Windows 10 Home Version 20H2. These were evaluated on a system with 16 GB RAM. In this experiment, we use TensorFlow - a major deep learning framework, to train and evaluate the model. This framework utilised the NVIDIA GeForce GTX 1060 3 GB on the evaluation system to effectively train the DL-based models using TensorFlow.

5.5. Settings of proposed framework

The proposed framework was adjusted is to evaluate different types of classifiers. Firstly, binary classifiers are built to detect attacks without identifying the class of such attacks. In this work, we apply global XAI methods (SHAP, PDP) to study the most important features in each dataset and select the most relevant features to build a more efficient model. Secondly, we build one-vs-all classifiers to detect each class of attack separately. After obtaining results from fitted one-vs-all classifiers, the proposed framework will generate both local and global explanations, giving insights into each feature's importance and the functioning of models. Lastly, we build target-vs-normal classifiers that distinguish between a targeted attack class and the normal traffic. This will help domain experts analyse the distinctive characteristics of each attack class or attack type.

We established the models based on fully connected networks of LSTM with ReLU as activation functions. The input dimension of each classifier depends on the dataset used as it equals the number of input features. Each classifier has 3 hidden layers, and each hidden layer contains 50 neurons. The output dimension of each classifier is 2. The TensorFlow framework is used to build the intrusion detection models effectively. We use Adam optimiser, the learning rate of 0.001 and the dropout rate of 0.1. The batch size is 32, with a total of 10 epochs.

5.6. Binary classifiers

We trained binary classifiers on the three datasets using the complete set of input features. Table 3 shows the evaluation indicators obtained from these classifiers. These values indicated that the performance of the models is sufficient to apply the proposed XAI framework and study the relevant features that models have learned to identify malicious activities. From the fitted model, the framework calculates the importance score of each feature using SHAP and PFI. The most important features of attacks that the model has learned from the three datasets are shown in Fig. 3.

As shown in Fig. 3, lists of the top 20 important features that SHAP and PFI generate are pretty different in all datasets. The outputs from PFI are easier to understand since each feature has a simple numeric value that indicates its overall impact on the model's decision. Therefore, the comparisons between features are relatively simple, which makes PFI's lists suitable for lay-users to understand. However, explanations generated by PFI are so simple that they only calculate the medium permutation importance without showing how much each features matters. This would lead to drawbacks in the case of features that have a significant impact on a few predictions but no impact in general or features that have an average effect on all predictions.

SHAP presents the list of the most important features by plotting positions on the graphs shown in Figs. 3a, 3c and 3e. We need to analyse each dot to understand the plot since each dot represents an instance in the dataset. The dot's vertical position indicates

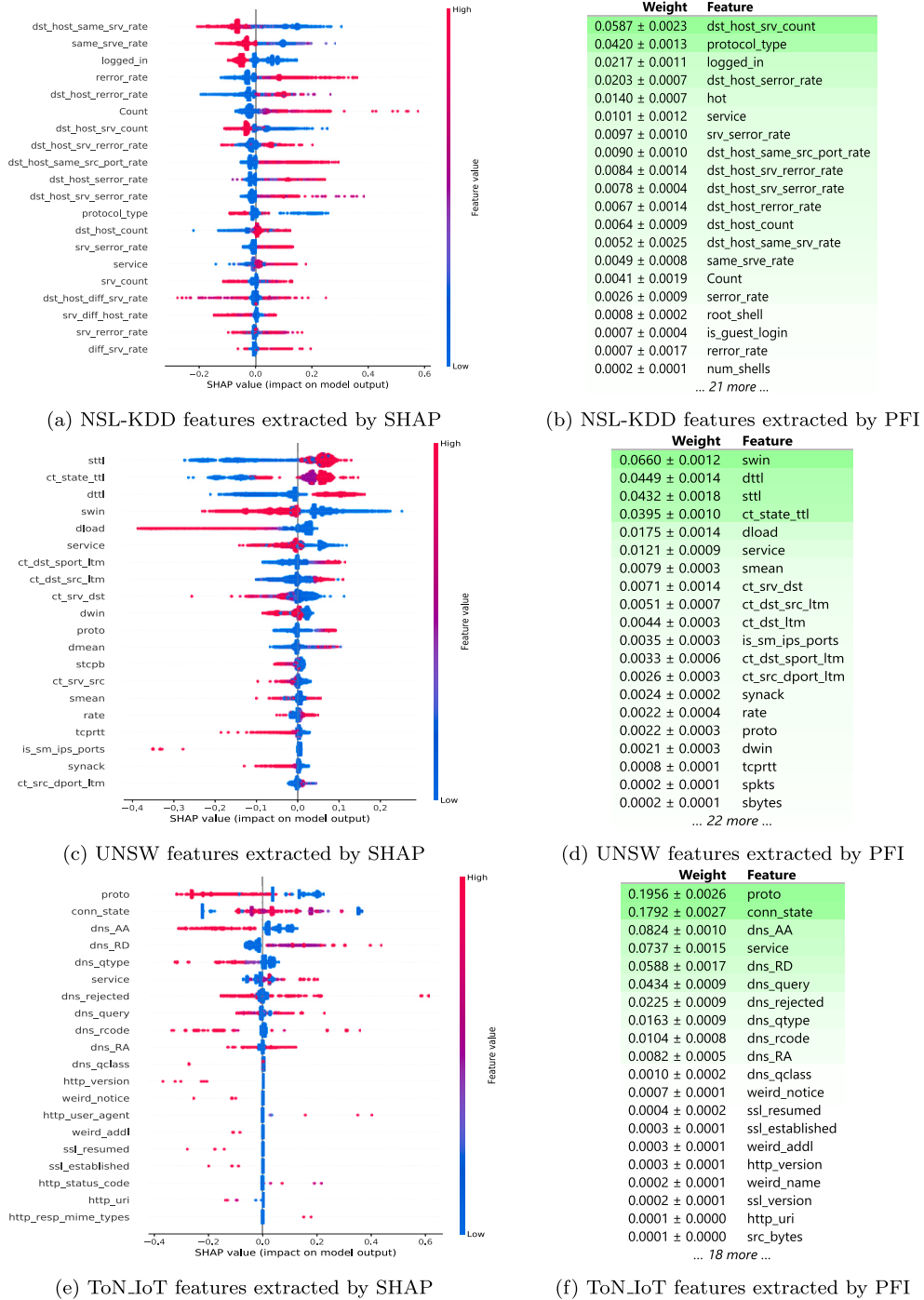


Fig. 3. Top 20 relevant features of attacks that binary classifiers learned from the three datasets (extracted by SHAP and PFI).

the feature it depicts; its horizontal position refers to the impact of that value on the model's prediction. The colours of the dot represent the value of that feature for that instance of the dataset, indicating whether it is high, medium or low (red, purple or blue, respectively). For example, in Fig. 3a, a high value of the feature 'Count' increased the probability that the activity is attack by 20% to 60%. SHAP gives a different view of feature importance and shows how each features matters. Therefore, it mitigates the major disadvantage of PFI's explanations that we discussed above.

Evaluation indicators in Table 4 show that when the classifiers are trained on subsets of input features, their performance only changes slightly and even gets better in a few cases. Models trained on the PFI subset of UNSW-NB15 and NSL-KDD datasets get higher recall and accuracy, respectively. Models trained on the SHAP subset of ToN_IoT and NSL-KDD datasets get higher precision.

Table 4
Results about Binary Classifiers.

Dataset	Accuracy	Precision	Recall	F1-score
NSL-KDD with SHAP features	0.787	0.970	0.646	0.776
NSL-KDD with PFI features	0.817	0.963	0.706	0.815
UNSW-NB15 with SHAP features	0.864	0.808	0.987	0.889
UNSW-NB15 with PFI features	0.859	0.801	0.990	0.885
ToN_IoT with SHAP features	0.872	0.790	0.866	0.826
ToN_IoT with PFI features	0.873	0.787	0.876	0.829

Table 5
Results of Binary Classifiers with Combined Set of Features.

Dataset	Accuracy	Precision	Recall	F1-score
NSL-KDD with combined features	0.787	0.971	0.645	0.775
UNSW-NB15 with combined features	0.858	0.797	0.994	0.885
ToN_IoT with combined features	0.873	0.785	0.879	0.829

Table 6
Training time and detection time taken (using different set of features).

	Original	SHAP	PFI	Combined
NSL-KDD training time (ms)	88801	87006	87520	87815
Total detection time (ms)	507	434	448	470
UNSW-NB15 training time (ms)	128519	123377	122447	121950
Total detection time (ms)	1444	1268	1336	1292
ToN_IoT training time (ms)	257544	250684	252824	259633
Total detection time (ms)	1664	1458	1508	1493

The sets of input features collected in these three datasets are not yet optimal to detect attack activities. Therefore, we combine the most important features extracted by SHAP and PFI (Fig. 3) and use them to re-train the binary classifier to detect attacks without altering the model's parameters.

The performance metrics of the binary classifier trained on each of the three selected datasets are provided in Table 5. As can be seen in the table, the performance of the models trained on the combined features selected by SHAP and PFI is comparable to the baseline performance and the results of Table 4. Although accuracy seems to have slightly dropped for the models trained on the combined subsets of UNSW-NB15 and NSL-KDD datasets, their recall and precision have increased, respectively. The model trained on the TON_IoT dataset has displayed a stable accuracy and F1-score, with a modest increase in precision and recall.

Therefore, it is shown that global explanations can be leveraged to reason the response of a classifier. Additionally, it is shown that, by selecting the most impactful features prior to training, a model's performance can be slightly improved, especially the precision and recall statistics. We also measure the time taken when using different feature sets for training and evaluating the models, as shown in Table 6. Using the set of features extracted by SHAP reduces training and detection time in all three datasets without significantly lowering the model's performance. The training time tends to be reduced much more if its value is already high (when using the original features). Therefore, using SHAP features would reduce the network overhead and computational resources without lowering the effectiveness of IDS in IoT networks.

5.7. One-vs-all classifiers

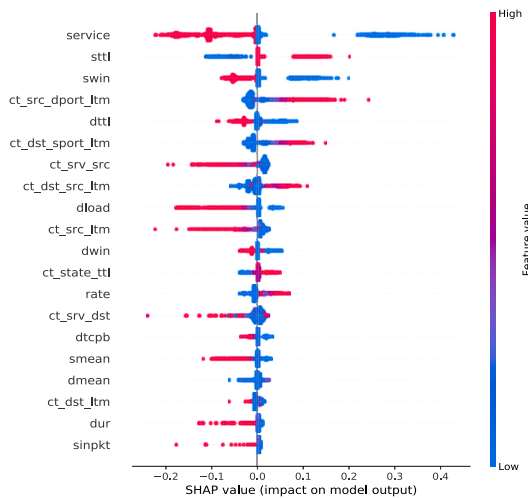
The next stage is to build one-vs-all classifiers to detect a specific class of attack and apply both global and local XAI methods. We expect that the explanations from the proposed framework would provide insight into the functioning of the models and help security experts study the characteristics of each cyber attack class. After building classifiers to detect individual attack classes, it is observed that the model can well detect a few attack classes, whilst the performance of classifiers in detecting other classes is insufficient for applying the proposed framework. Results from training one-vs-all classifiers are shown in Table 7.

Among all classes of cyber attack in Table 7, the models can obtain the best performance on *Generic*. Therefore, we apply the XAI methods in analysing the model's decision in detecting *Generic*. Based on the explanations generated by both SHAP and PFI in Fig. 4, the following features are considered as the most important features to detect a *Generic* attack:

- *service*: Low values of *service* feature would not affect the model's decision, but in many other cases, they increase the probability that the model identifies the records as *Generic* attack by 25% - 40%. List of services in ascending order of value: *http*, *FTP*, *SMTP*, *POP3*, *DNS*, *SNMP*, *SSL*, *DHCP*, *IRC*, *radius*, *SSH*, *none*.
- *swin*: It refers to the values of source TCP window advertisement. Low values of *swin* feature would not affect the model's decision, but in many other cases, they increase the probability of a prediction as *Generic* attack by 10% - 20%.

Table 7
Results about one-vs-all classifiers.

Dataset	Attack class	Accuracy	Precision	Recall	F1-score
NSL-KDD	DoS	0.931	0.958	0.829	0.889
	Probe	0.946	0.819	0.636	0.716
	U2R	0.690	0.923	0.747	0.826
	R2L	0.680	0.949	0.720	0.819
UNSW-NB15	Exploits	0.920	0.763	0.596	0.669
	Generic	0.991	0.999	0.962	0.980
	Reconnaissance	0.979	0.812	0.664	0.731
	Analysis	0.899	0.899	0.951	0.925
	Shellcode	0.794	0.817	0.987	0.894
	DoS	0.840	0.799	0.943	0.866
	Worms	0.850	0.780	0.893	0.833
	Fuzzers	0.799	0.760	0.995	0.862
	Backdoors	0.720	0.670	0.853	0.751
ToN_IoT	DDoS	0.987	0.822	0.902	0.859
	XSS	0.969	0.629	0.736	0.678
	Scanning	0.776	0.796	0.894	0.842
	Backdoor	0.750	0.870	0.901	0.885



(a) UNSW-NB15 features extracted by SHAP

Weight	Feature
0.1752 ± 0.0009	service
0.0520 ± 0.0009	swin
0.0474 ± 0.0010	dttl
0.0354 ± 0.0009	sttl
0.0264 ± 0.0006	dwin
0.0122 ± 0.0005	smean
0.0109 ± 0.0008	dload
0.0044 ± 0.0003	dmean
0.0044 ± 0.0003	ct_src_ltm
0.0040 ± 0.0002	dtepb
0.0036 ± 0.0006	dur
0.0035 ± 0.0002	rate
0.0028 ± 0.0002	sinpkt
0.0017 ± 0.0001	is_ftp_login
0.0016 ± 0.0002	is_sm_ips_ports
0.0011 ± 0.0001	sjit
0.0005 ± 0.0001	dinpkt
0.0004 ± 0.0001	ct_srv_src
0.0004 ± 0.0002	ct_srv_dst
0.0003 ± 0.0001	stcpb
... 22 more ...	

(b) UNSW-NB15 features extracted by PFI

Fig. 4. Top 20 important features of *Generic* attack class that models have learned from UNSW-NB15.

- *dttl*: This feature measures the time to live of destination to source connections. Low values of *dttl* feature would slightly increase the chance that the record are detected as *Generic* attack.
- *sttl*: This feature measures the time to live of source to destination connections. In contrast to *dttl*, high values of *sttl* feature would increase the chance that the record is the *Generic* attack by 10% - 20%.
- *ct_src_dport_ltm*: This feature refers to the number of connections of the same source address and the destination port in 100 connections according to the last time. Interestingly, SHAP differs from PFI in computing the effect of this feature on the model's decisions. In this case, PFI hinders the feature's impact because PFI does not demonstrate how much this feature matters. PFI only computes the medium permutation importance and neglects data records in which this feature has a significant impact.

SHAP can also be used to generate individual explanations. Fig. 5 shows a data record that is classified as *Generic* attack. The model's decision is decomposed into the sum of effects of each feature value. In this specific data instance, *service* = 0.005015 (dns), *sttl* = 0.9961 (254) and *ct_src_dport_ltm* = 0.3 (16) are the three features that contribute the most to the final decision of the model.

To further analyse the effects of these features on the overall decision or individual decisions of the model, we apply PDP and ICE as shown in Fig. 6a and 6a. ICE lines are blue, and the average of all blue lines is yellow lines, also known as PDP. These methods demonstrate the model's prediction changes when a feature changes. Fig. 6a tell us that the increase in value of *service* feature would lead to decrease in the probability that the model classify an activity as *Generic* attack. Moreover, it can be divided into two groups of instances depending on how much the value of *service* feature affects the model's prediction. One group slightly decreases the chance of a *Generic* attack by only 10%, whilst the other group significantly decreases such chance by 90% and up to 100%.

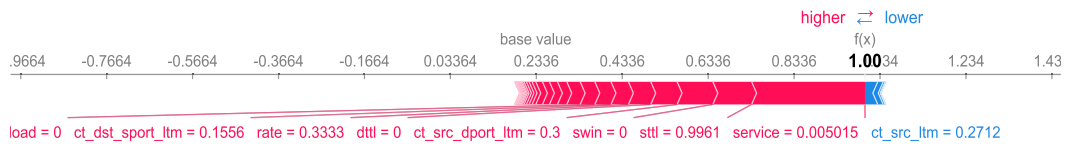
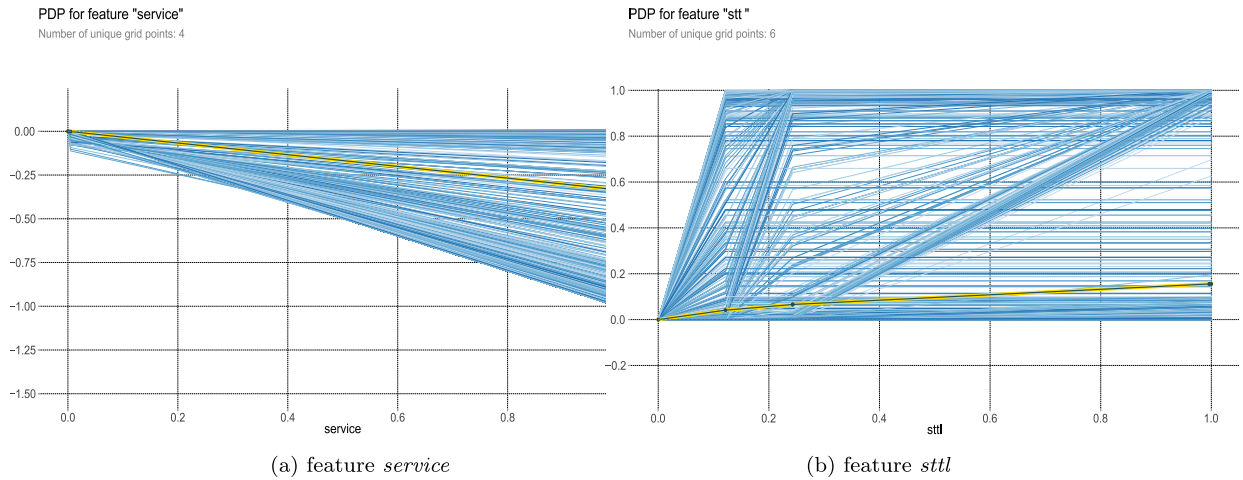
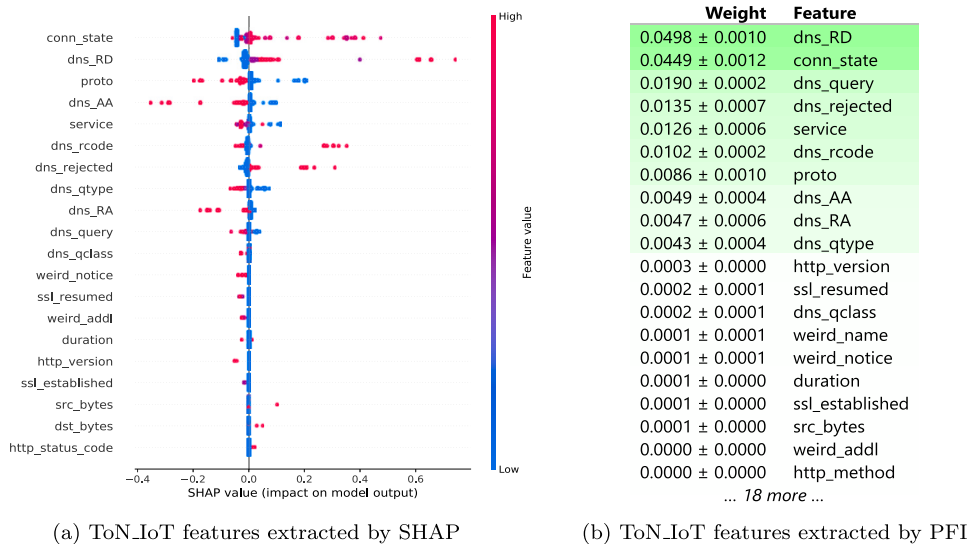
Fig. 5. A local explanation on the *Generic* attack class.Fig. 6. PDP and ICE for features *service* and *sttl* feature in detecting *Generic* attack.Fig. 7. Top 20 important features of *DDoS* attack class that models have learned from ToN_IoT.

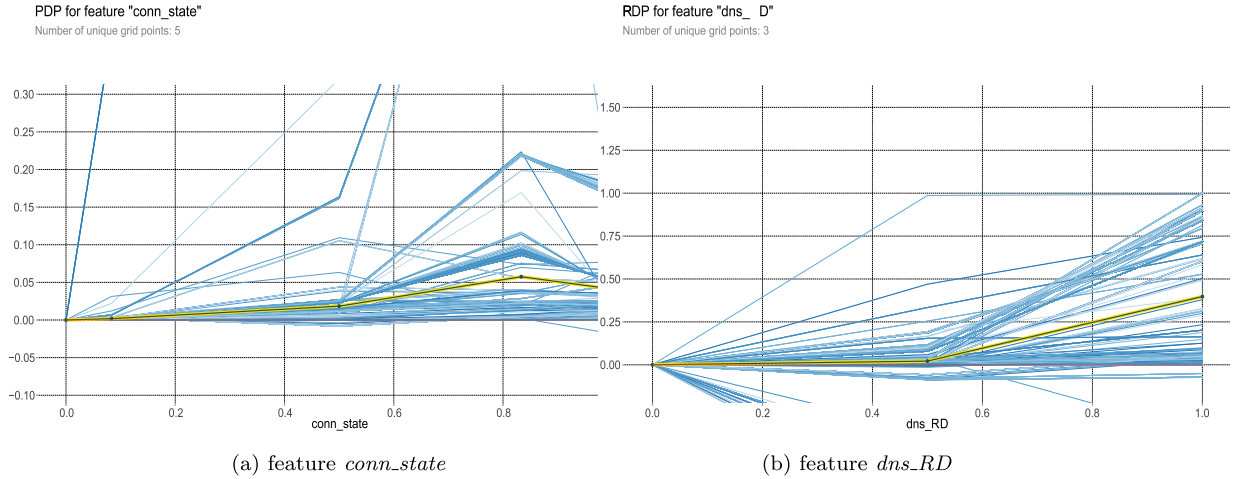
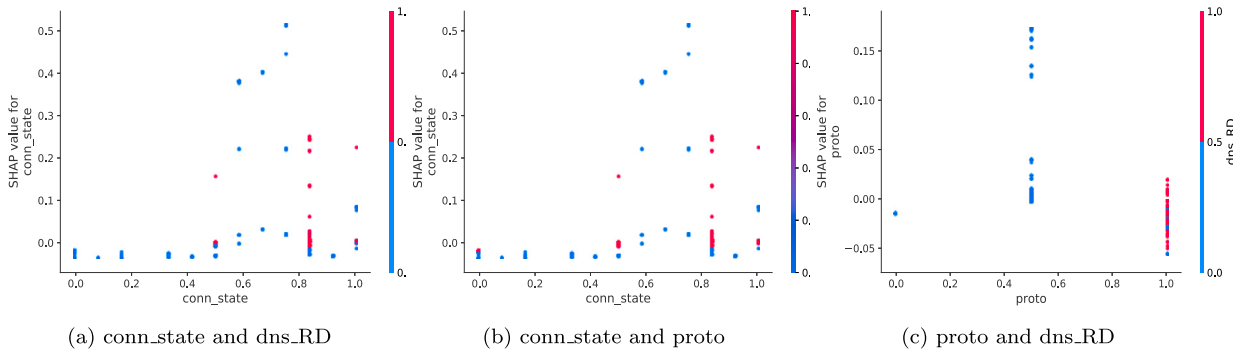
Fig. 6b is more complex as the data instances can be divided into 4 groups. Data records in the first and second groups would be classified as *Generic* attack if the value of *sttl* feature is around 0.1 and 0.25, respectively. Whilst data records in the third group are classified as an attack if the value of *sttl* feature reaches its maximum, the effect of *sttl* feature on data instances' decisions in the fourth group is very small.

The one-vs-all classifier that detects *DDoS* attack in ToN_IoT dataset also perform well, which is shown in Table 7. This good performance allows us to further analyse the results by using XAI methods as we had used above to study *Generic* attack. Fig. 7 show the list of the 20 most important features extracted by SHAP and PFI from the ToN_IoT dataset. Both SHAP and PFI extract some notable features, including:

- *conn_state*: this feature is a string that refers to the state of connections, such as S0 (connection without replay), S1 (connection established), and REJ (connection attempt rejected). Higher values of this feature would lead to higher probability that the data



Fig. 8. A local explanation on the DDoS attack class.

Fig. 9. PDP and ICE for features *conn_state* and *dns_RD* feature in detecting DDoS attack.Fig. 10. Dependence contribution plots of features *proto*, *dns_RD* and *conn_state*.

instances are classified as DDoS attacks. Symbols of *conn_state* feature in ascending order of value: *OTH*, *REJ*, *RSTO*, *RSTOSO*, *RSTR*, *RSTRH*, *S0*, *S1*, *S2*, *S3*, *SF*, *SH*, *SHR*

- *dns_RD*: this feature has boolean data type that indicates the recursion desire of DNS. After applying label encoding and scaler, the data pre-processor assigns value of “-” as 0, value of *False* as 1 and value of *True* as 2. Same as *conn_state* feature, higher value of *dns_RD* increase the chance that an instance is classified as DDoS attack.

Fig. 8 demonstrates an individual explanation for a single data instance that is correctly classified as DDoS attack by the model. The value “0.5833” of feature *conn_state* contributes greatly to the final decision, which means that if *conn_state* equals to *S1* the activity is likely classified as DDoS attack. Moreover, *proto* = 0.5 (TCP) and *service* = 0 (none) also contribute to such decision. Although *dns_RD* = 0 (none) decreases the chance of a DDoS attack, the model has computed that overall probability equals to 0.78. Therefore, this is a DDoS attack.

Fig. 9a and 9b shows PDP and ICE for the two features *conn_state* and *dns_RD*. Overall, high values of both of these features would increase the chance of a DDoS attack. Specifically, the feature *dns_RD* significantly increase such probability if it reaches maximum value. However, compared to Fig. 6a and 6b, there are many data instances in Fig. 9a and 9b that do not follow the rules mentioned above, which signifies hidden relationships between input features. This requires us to further analyse the results and demonstrate any existing correlations between features that can affect the final decision of the model.

Fig. 10 demonstrates SHAP dependence plot for the three most important features consisting of *conn_state*, *dns_RD* and *proto*. Each dot on these plots represents a data instance in the dataset. The horizontal position indicates the value of the feature on the x-axis,

Table 8
Performance of comparable anomaly-based models.

Dataset	Metrics	GAA	MCA	TANN
NSL-KDD	DR	0.987	0.963	0.912
	FPR	0.004	0.049	0.094
UNSW-NB15	DR	0.949	0.90	0.875
	FPR	0.056	0.068	0.123

and the colour represents the value of other features shown on the right column. The vertical position refers to the SHAP value that indicates how much these values of the two features affect the model's decision. Before analysing these plots, there are three different types of the protocol used in ToN_IoT, including ICMP (labelled as "0"), TCP (labelled as "0.5") and UDP (labelled as "1").

Fig. 10a and 10b are quite similar in terms of distributions. Both plots observe high value of SHAP when the value of *conn_state* is in range of [0.5, 1]. Specifically, SHAP value is high when the value of *conn_state* is in range of [0.6, 0.8] and the values of *dns_RD* or *proto* are low. Whilst, high value of *dns_RD* or *proto* would significantly increase SHAP value when *conn_state*'s value is smaller than 0.6 or higher than 0.8. Fig. 10c shows that all detected DDoS attacks use the UDP protocol.

5.8. Relationship between feature values and attacks

Applying local and global XAI methods on one-vs-all classifiers provides us with interesting results that reflect insight into the datasets and characteristics of each attack class. The relationships between the input features and individual class of attack are demonstrated through visualisation using our proposed framework. These explanations are intuitive and would be easy to understand for different XAI recipients with various levels of knowledge about cyber security or artificial intelligence. Therefore, the SPIP framework can effectively enhance the benefits that XAI models could bring to audiences.

The experiments in this paper have shown a strong relationship between input features and attack classes. By applying the proposed framework on well-performing one-vs-all classifiers, we can analyse such relationships and the decision process of the AI-based models. For example, the framework studies that features *conn_state*, *dns_RD* and *proto* have strong relationships with DDoS attacks, which matches the real characteristics of DDoS attacks. This indicates that the classifier has given valid predictions as it has studied the right patterns of the particular attack class. Moreover, this has proven that the explanations generated by the SPIP framework can be used to effectively enhance the interpretability and explainability of AI-based intrusion detection models.

5.9. Comparisons with peer techniques & discussions

To examine the performance of the proposed SPIP-based IDS framework, three comparable anomaly-based detection techniques were leveraged, namely Geometric Area Analysis (GAA) [47], Multivariate Correlation Analysis (MCA) [48], Triangle Area Nearest Neighbours (TANN) [49]. From these anomaly techniques, the TANN and MCA were initially designed for DoS attack detection and thus resulted in models with increased performance when tasked with detecting such attack types. On the other hand, GAA leverages mixture models to detect anomalies based on pre-defined baselines. Table 8 depicts the performance of these methods, compared to, these models achieve high detection rates, however, their FPR is considerable and thus not suited to real-world application. On the other hand, our binary classifiers produce more stable results, evidenced by the high precision and recall metrics.

There are several distinct and significant benefits to applying the proposed framework. To begin with, the model assists developers and researchers in evaluating the quality, analysing and detecting any bias in the dataset. This is done by combining the foundation knowledge about individual attack classes and the explanations generated from the SPIP framework. SPIP's reasonable explanations support the interpretability of the IDS. It provides the recipient with the demonstration of relationships between features' value and the prediction and relationships between input features.

Moreover, the framework is post-hoc, which can be applied to any model regardless of the underlying algorithms. SPIP evaluates the quality of the collected input features in detecting different attack classes. Each attack class has specific characteristics shown in the features' value. A quality set of input features would demonstrate these characteristics, enhancing trust in the models' decisions. Therefore, outputs from SPIP can help researchers evaluate the input features and discover novel features to collect from network activities. Zero-day attacks exploit unknown vulnerabilities in the system; therefore, it is difficult to detect and analyse them. AI-based IDS can detect zero-day attacks, and the SPIP framework can explain such decisions. SPIP's outputs would enhance the investigation process by helping security experts discover these unknown attacks' specific characteristics.

The SPIP framework has certain limitations that may hinder its effectiveness. Strong IDS performance requires a comprehensive training set with valuable collected features from network activities. This is a significant challenge in building an effective IDS. It is impossible to construct a dataset that involves all normal and malicious behaviours in a heterogeneous environment of IoT networks. Moreover, many existing datasets collect an incomplete set of features, and network information is captured without including both headers and payloads [50]. However, the SPIP framework can support security experts in collecting more relevant features from the network traffic, enhancing classifiers' performance. The generated explanations are limited to presenting notable features and their correlations without determining the security vulnerabilities that the attack exploits. However, security experts would utilise those results to discover and demonstrate the existing vulnerabilities based on their knowledge about the victim system and the notified attacks.

6. Conclusion & future work

This paper has proposed the SPIP framework, developed to evaluate explainable DL algorithms for IDS deployments in IoT environments. The SPIP framework generates both local and global explanations and combines various XAI and DL methods, including Long Short-Term Memory (LSTM), Shapley Additive Explanations (SHAP), Permutation Feature Importance (PFI), Individual Conditional Expectation (ICE) and Partial Dependence Plot (PDP). The generated explanations by our framework are intuitive and easy for audiences of different technical levels to understand. The SPIP framework extracts a customised set of input features that outperforms the original set in the three datasets, NSL-KDD, UNSW-NB15 and ToN_IoT. More importantly, the relationships extracted by SPIP match specific attack classes' characteristics. The framework proposed in this paper would significantly enhance the utilisation of AI-based IDS in the cyber defence system.

The output of the proposed SPIP framework dramatically relies on the performance of intrusion detection models, which means that the performance of the framework suffers when coupled with a poorly constructed IDS. Moreover, the proposed framework cannot specifically identify the vulnerability that an attack class exploits. Future research can build a framework that can suggest potential vulnerabilities by studying the features of network traffic generated during cyberattacks.

CRediT authorship contribution statement

Marwa Keshk: Conceptualization, Methodology / Study design, Validation, Formal analysis, Data curation, Visualization, Writing, Editing.

Nam Pham: Conceptualization, Methodology / Study design, Validation, Formal analysis, Data curation, Visualization, Writing, Editing.

Nickolaos Koroniotis: Conceptualization, Methodology / Study design, Validation, Formal analysis, Data curation, Visualization, Writing, Editing.

Nour Moustafa: Conceptualization, Methodology / Study design, Validation, Formal analysis, Data curation, Writing, Funding acquisition, Visualization Editing, Supervision.

Benjamin Turnbull: Conceptualization, Methodology / Study design, Formal analysis, Writing, Editing, Visualization, Supervision.

Albert Y. Zomaya: Conceptualization, Methodology / Study design, Formal analysis, Writing, Visualization Editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgement

This research was supported under Australian Research Council's Discovery Early Career Researcher Award (DECRA) (project number, DE230100116).

References

- [1] Government Regulations in Cyber Security: Framework, Standards and Recommendations, *Future Gener. Comput. Syst.* 92 (2019) 178–188.
- [2] J. Asharf, N. Moustafa, H. Khurshid, E. Debie, W. Haider, A. Wahab, A review of intrusion detection systems using machine and deep learning in internet of things: challenges, solutions and future directions, *Electronics* 9 (7) (2020).
- [3] I.H. Sarker, M.H. Furhad, R. Nowrozy, Ai-driven cybersecurity: an overview, security intelligence modeling and research directions, *SN Comput. Sci.* 2 (3) (2021) 1–18.
- [4] M.M. Hassan, A. Gumaiei, A. Alsanad, M. Alrubaian, G. Fortino, A hybrid deep learning model for efficient intrusion detection in big data environment, *Inf. Sci.* 513 (2020) 386–396.
- [5] S. Mohammadi, H. Mirvaziri, M. Ghazizadeh-Ahsaei, H. Karimipour, Cyber intrusion detection by combined feature selection algorithm, *J. Inf. Secur. Appl.* 44 (2019) 80–88.
- [6] N. Moustafa, G. Misra, J. Slay, Generalized outlier Gaussian mixture technique based on automated association features for simulating and detecting web application attacks, in: *IEEE Transactions on Sustainable Computing*, 2018.
- [7] N. Moustafa, A new distributed architecture for evaluating ai-based security systems at the edge: network ton_iot datasets, *Sustain. Cities Soc.* 72 (2021) 102994.
- [8] G. Andresini, A. Appice, D. Malerba, Autoencoder-based deep metric learning for network intrusion detection, *Inf. Sci.* 569 (2021) 706–727.
- [9] H. Hindy, R. Atkinson, C. Tachtatzis, J.-N. Colin, E. Bayne, X. Bellekens, Utilising deep learning techniques for effective zero-day attack detection, *Electronics* 9 (10) (2020) 1684.
- [10] X. Kan, Y. Fan, Z. Fang, L. Cao, N.N. Xiong, D. Yang, X. Li, A novel iot network intrusion detection approach based on adaptive particle swarm optimization convolutional neural network, *Inf. Sci.* 568 (2021) 147–162.
- [11] C. Wu, A. Qian, X. Dong, Y. Zhang, Feature-oriented design of visual analytics system for interpretable deep learning based intrusion detection, in: *2020 International Symposium on Theoretical Aspects of Software Engineering (TASE)*, IEEE, 2020, pp. 73–80.

- [12] D.E. Denning, An intrusion-detection model, *IEEE Trans. Softw. Eng.* 2 (1987) 222–232.
- [13] A. Drewek-Ossowicka, M. Pietrolaj, J. Rumiński, A survey of neural networks usage for intrusion detection systems, *J. Ambient Intell. Humaniz. Comput.* 12 (1) (2021) 497–514.
- [14] T. Saranya, S. Sridevi, C. Deisy, T.D. Chung, M.K. Khan, in: *Performance Analysis of Machine Learning Algorithms in Intrusion Detection System: A Review*, in: *Procedia Computer Science*, vol. 171, Elsevier B.V., 2020, pp. 1251–1260.
- [15] J.D. Moore, W.R. Swartout, *Explanation in expert systems: a survey*, Tech. Rep., University of Southern California, Marina del Rey Information Sciences Inst., 1988.
- [16] M. Almiani, A. AbuGhazleh, A. Al-Rahayfeh, S. Atiewi, A. Razaque, Deep recurrent neural network for iot intrusion detection system, *Simul. Model. Pract. Theory* 101 (2020) 102031.
- [17] V.K. Rahul, R. Vinayakumar, K. Soman, P. Poornachandran, Evaluating shallow and deep neural networks for network intrusion detection systems in cyber security, in: *2018 9th International Conference on Computing, Communication and Networking Technologies, ICCNT 2018*, Institute of Electrical and Electronics Engineers Inc., 2018.
- [18] B. Zhou, Y. Sun, D. Bau, A. Torralba, Interpretable basis decomposition for visual explanation, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 119–134.
- [19] S. Anjomshoaie, K. Främling, A. Najjar, Explanations of black-box model predictions by contextual importance and utility, in: *International Workshop on Explainable, Transparent Autonomous Agents and Multi-Agent Systems*, Springer, 2019, pp. 95–109.
- [20] N. Seedat, V. Aharonson, Y. Hamzany, Automated and interpretable m-health discrimination of vocal cord pathology enabled by machine learning, in: *2020 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)*, IEEE, 2020, pp. 1–6.
- [21] T.R. Wood, C. Kelly, M. Roberts, B. Walsh, An interpretable machine learning model of biological age, *F1000, Research* 8 (17) (2019) 17.
- [22] A. Andalib, V.T. Vakili, An autonomous intrusion detection system using an ensemble of advanced learners, in: *2020 28th Iranian Conference on Electrical Engineering (ICEE)*, 2020, pp. 1–5.
- [23] S.R. Islam, W. Eberle, S.K. Ghafoor, A. Siraj, M. Rogers, Domain knowledge aided explainable artificial intelligence for intrusion detection and response, *arXiv:1911.09853*, 11 2019.
- [24] D.L. Marino, C.S. Wickramasinghe, M. Manic, An adversarial approach for explainable ai in intrusion detection systems, in: *IECON 2018 - 44th Annual Conference of the IEEE, Industrial Electronics Society*, 2018, pp. 3237–3243.
- [25] K. Čyras, A. Rago, E. Albini, P. Baroni, F. Toni, Argumentative xai: a survey, in: Z.-H. Zhou (Ed.), *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21, International Joint Conferences on Artificial Intelligence Organization*, 2021, pp. 4392–4399, Survey Track.
- [26] N. Nobani, F. Mercorio, M. Mezzananza, Towards an explainer-agnostic conversational xai, in: Z.-H. Zhou (Ed.), *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21, International Joint Conferences on Artificial Intelligence Organization*, 2021, pp. 4909–4910, Doctoral Consortium.
- [27] M.T. Keane, E.M. Kenny, E. Delaney, B. Smyth, If only we had better counterfactual explanations: five key deficits to rectify in the evaluation of counterfactual xai techniques, *arXiv preprint arXiv:2103.01035*, 2021.
- [28] H. Li, F. Wei, H. Hu, Enabling Dynamic Network Access Control with Anomaly-Based IDS and SDN, *Association for Computing Machinery*, New York, NY, USA, 2019.
- [29] P. Werbos, Backpropagation through time: what it does and how to do it, *Proc. IEEE* 78 (10) (1990) 1550–1560.
- [30] Y. Bengio, P. Simard, P. Frasconi, Learning long-term dependencies with gradient descent is difficult, *IEEE Trans. Neural Netw.* 5 (2) (1994) 157–166.
- [31] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* 9 (8) (1997) 1735–1780.
- [32] S. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, *arXiv preprint arXiv:1705.07874*, 2017.
- [33] D. Apley, Visualizing the effects of predictor variables in black box supervised learning models, *arXiv preprint arXiv:1612.08468*, 2016.
- [34] A. Fisher, C. Rudin, F. Dominici, All models are wrong, but many are useful: learning a variable's importance by studying an entire class of prediction models simultaneously, *J. Mach. Learn. Res.* 20 (177) (2019) 1–81.
- [35] Q. Zhao, T. Hastie, Causal interpretations of black-box models, *J. Bus. Econ. Stat.* 39 (1) (2021) 272–281.
- [36] L.S. Shapley, A value for n-person games, *Contrib. Theory Games* 2 (28) (1953) 307–317.
- [37] M.T. Ribeiro, S. Singh, C. Guestrin, “Why should I trust you?” explaining the predictions of any classifier, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [38] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32.
- [39] J.H. Friedman, Greedy function approximation: a gradient boosting machine, *Ann. Stat.* (2001) 1189–1232.
- [40] C. Molnar, *Interpretable machine learning*, <https://christophm.github.io/interpretable-ml-book/>, 2019.
- [41] B. Mahbooba, M. Timilsina, R. Sahal, M. Serrano, Explainable artificial intelligence (xai) to enhance trust management in intrusion detection systems using decision tree model, *Complexity* (2021) 2021.
- [42] P.N. Bahrami, A. Dehghantanha, T. Dargahi, R.M. Parizi, K.-K.R. Choo, H.H. Javadi, Cyber kill chain-based taxonomy of advanced persistent threat actors: analogy of tactics, techniques, and procedures, *J. Inf. Process. Syst.* 15 (4) (2019) 865–889.
- [43] M. Tavallaei, E. Bagheri, W. Lu, A.A. Ghorbani, A detailed analysis of the kdd cup 99 data set, in: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009, pp. 1–6.
- [44] L. Dhanabal, S. Shantharajah, A study on nsl-kdd dataset for intrusion detection system based on classification algorithms, *Int. J. Adv. Res. Comput. Commun. Eng.* 4 (6) (2015) 446–452.
- [45] N. Moustafa, J. Slay, Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set), in: *2015 Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6.
- [46] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A. Anwar, Ton_iot telemetry dataset: a new generation dataset of iot and iiot for data-driven intrusion detection systems, *IEEE Access* 8 (2020) 165130–165150.
- [47] N. Moustafa, J. Slay, G. Creech, Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks, *IEEE Trans. Big Data* 5 (4) (2017) 481–494.
- [48] Z. Tan, A. Jamdagni, X. He, P. Nanda, R.P. Liu, A system for denial-of-service attack detection based on multivariate correlation analysis, *IEEE Trans. Parallel Distrib. Syst.* 25 (2) (2013) 447–456.
- [49] C.-F. Tsai, C.-Y. Lin, A triangle area based nearest neighbors approach to intrusion detection, *Pattern Recognit.* 43 (1) (2010) 222–229.
- [50] N. Moustafa, J. Hu, J. Slay, A holistic review of network anomaly detection systems: a comprehensive survey, *J. Netw. Comput. Appl.* 128 (2019) 33–55.